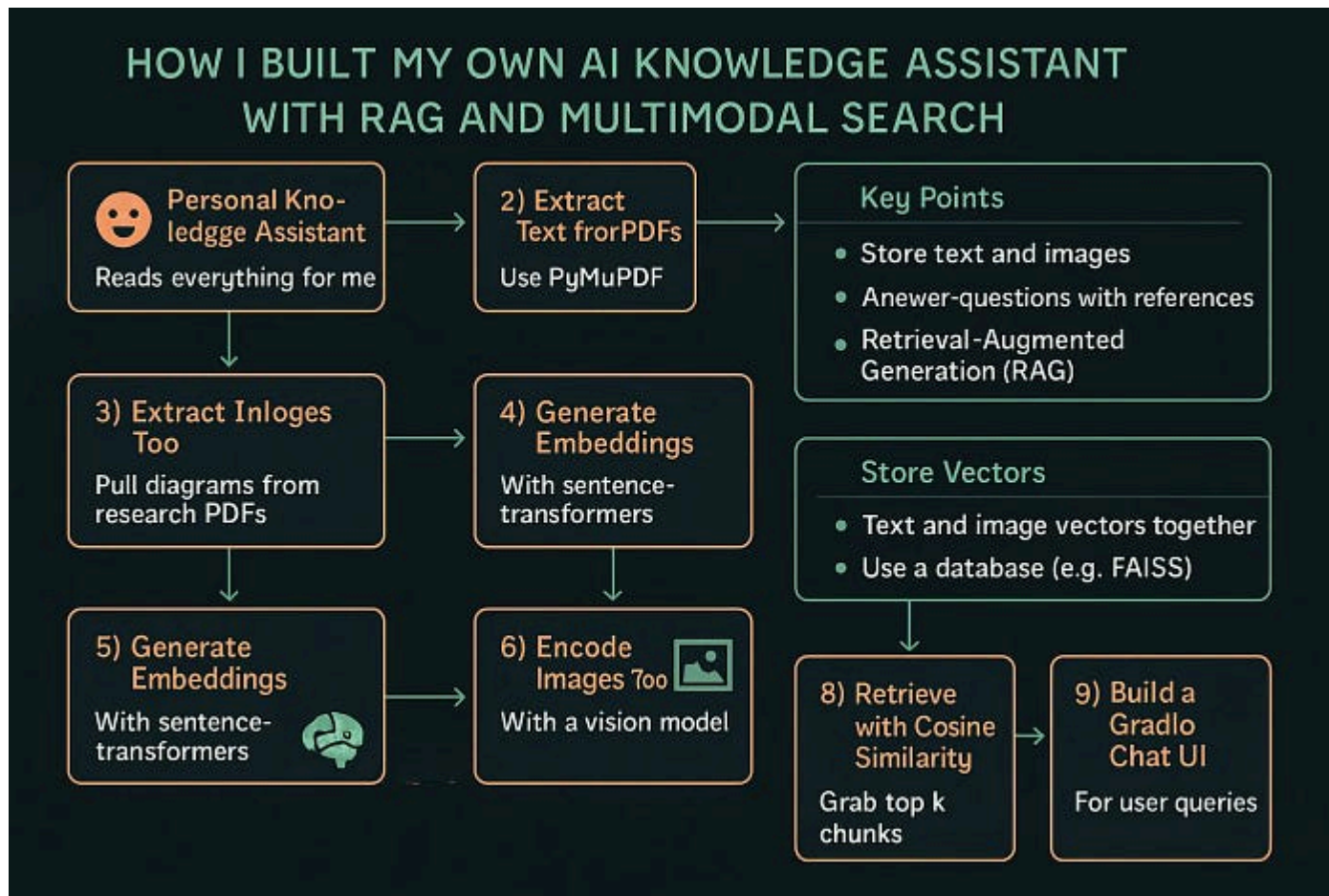


[< Go to the original](#)

How I Built My Own AI Knowledge Assistant with RAG and Multimodal Search

**Mariyam Mahmood**

Follow



Data Science Collective a11y-light ~3 min read · August 4, 2025 (Updated: August 4, 2025) · Free: No

1) Why a Personal Knowledge Assistant?

I hoard PDFs. Articles. Screenshots. Notes. I forget 99% of them. So last year, I built a tool that reads everything for me. The goal:

- Store everything (text, images)
- Ask questions later *and get exact answers with references.*

This is Retrieval-Augmented Generation (RAG). Let's build one.

2) Extract Text from PDFs with PyMuPDF

Copy

```
import fitz # PyMuPDF
pdf = fitz.open("document.pdf")
all_text = ""
```

```
print(doc_text[:500])
```

Why `fitz` ? Fast, reliable, finds text where `PyPDF2` fails.

3) Extract Images Too

Most of my research PDFs are half diagrams. So we pull those too.

Copy

```
for page_index in range(len(pdf)):
    page = pdf[page_index]
    images = page.get_images(full=True)
    for img_index, img in enumerate(images):
        xref = img[0]
        base_image = pdf.extract_image(xref)
        img_bytes = base_image["image"]
    with open(f"image_{page_index}_{img_index}.png", "wb") as f:
        f.write(img_bytes)
```

4) Chunk the Text for Embedding

Chunks make retrieval sharper. 500–1,000 tokens is typical.

```
def chunk_text(text, chunk_size=500):  
    words = text.split()  
    chunks = []  
    for i in range(0, len(words), chunk_size):  
        chunk = " ".join(words[i:i+chunk_size])  
        chunks.append(chunk)  
    return chunks  
chunks = chunk_text(all_text)
```

5) Generate Embeddings with sentence-transformers

Copy

```
from sentence_transformers import SentenceTransformer  
model = SentenceTransformer("all-MiniLM-L6-v2")  
embeddings = model.encode(chunks)  
print(embeddings.shape)
```

This maps your text into vector space. Same for queries later.

6) Encode Images Too

Some embedding models do both text & vision.

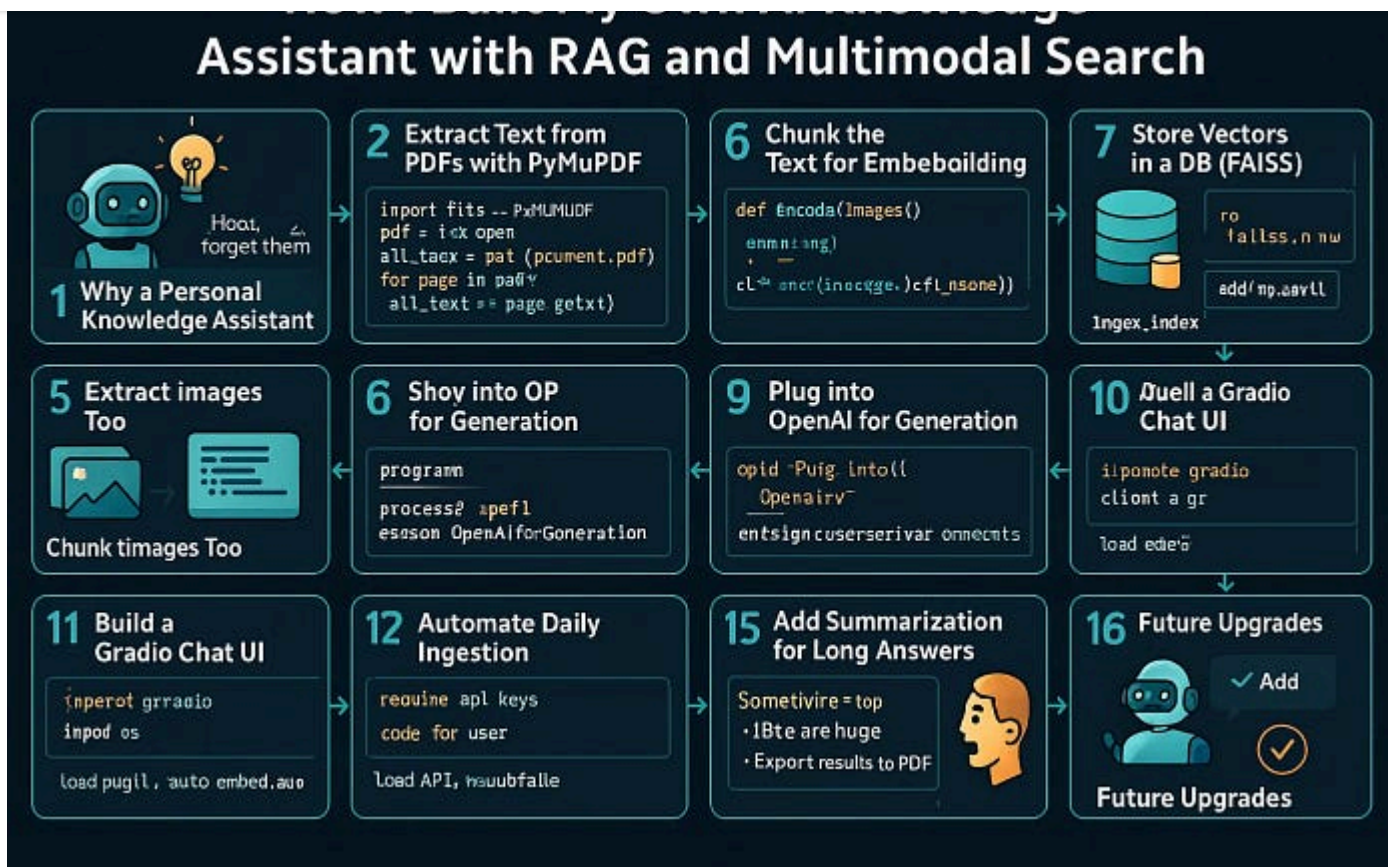
```
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
image = Image.open("image_0_0.png")
inputs = processor(images=image, return_tensors="pt")
image_embeds = model.get_image_features(**inputs)
```

7) Store Vectors in a DB (FAISS)

Copy

```
import faiss
import numpy as np
dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings).astype("float32"))
```

Text & image vectors live together search them the same way.



8) Retrieve with Cosine Similarity

Copy

```
query = "What is transformer attention?"
query_embedding = model.encode([query])
```

Grab top k chunks, feed them to your LLM prompt.

9) Plug into OpenAI for Generation

Copy

```
from openai import OpenAI
client = OpenAI()
system_prompt = """
You are a helpful assistant.
"""
user_prompt = f"""
Answer this based on context:
{chunks[I[0][0]]}
Question:
{query}
"""
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ]
)
print(response.choices[0].message)
```

10) Show Source Snippets in Response

Always include the source chunk. Helps you verify. Show page numbers, filenames, or even highlight in the PDF.

11) Build a Gradio Chat UI

Copy

```
import gradio as gr
def answer(query):
    # Embed query, search FAISS, feed to LLM - return result
    return "Answer goes here."
gr.ChatInterface(fn=answer).launch()
```

12) Automate Daily Ingestion

New PDFs in /dropbox ? Add a daily cron:

Copy

```
0 3 * * * python ingest.py
```


13) Secure API Keys with .env

Copy

```
from dotenv import load_dotenv
import os
load_dotenv()
openai_api_key = os.getenv("OPENAI_API_KEY")
```

14) Dockerize the Whole Thing

Copy

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

15) Add Summarization for Long Answers

Sometimes your top k hits are huge. Re-summarize.

```
context = ".join([chunks[i] for i in i[0]])\n\nprompt = f\"\"\"\nPlease summarize this for me:\n{context}\n\"\"\"\n\nresponse = client.chat.completions.create(\n    model="gpt-4o-mini",\n    messages=[{"role": "user", "content": prompt}]\n)\nprint(response.choices[0].message)
```

16) Future Upgrades

✅ Add user memory (who asked what) ✅ Use voice input + TTS for Q&A ✅

Export results to Notion ✅ Plug in live web search too

Final Thought

This personal RAG bot runs 24/7. It reads everything. It remembers. It explains.

#data-science #coding #programming #technology #python